

Retraction Is an Operation

Flyxion
Independent Researcher

October 2026

Abstract

Correction, deletion, withdrawal, and revocation are usually treated as informal variants of a single act of taking something back. This essay treats them as distinct operations on a history and asks what each does to three independent properties: what is presently authoritative, what remains recoverable, and what remains auditable. The organizing result is that these properties come apart. Preserving a commitment to an entry preserves neither its content nor its former authority, and each of the four operations moves the three properties differently. The central theorem is conditional and elementary once the semantics are fixed: if the history is append-only, every retraction is an entry, and the authority rule respects withdrawal, then a retracted claim leaves the authoritative set while every earlier state remains replayable. The essay then identifies the cases in which these conditions fail, namely dependence between claims, demands for erasure, and unannounced rewriting, and shows that controlled loss of content can be separated from undetectable alteration of history.

1 Motivation and Scope

Words for taking something back are used interchangeably in ordinary speech and in institutional practice, although the acts they name have different consequences. A journal retraction commonly leaves the article accessible with a notice attached, so the article remains citable as a fact about the literature while ceasing to be authoritative as a finding. A force-pushed repository replaces one sequence of commits with another, so that a clone made earlier and a clone made later no longer describe the same history. A redacted record keeps its place in a file while withholding what it said, and a revoked credential stops being accepted for future use while leaving open whether anything done under it earlier should still be trusted. A withdrawn public statement may remain in circulation, be removed from its original venue, or be superseded by a new statement, and each of these affects those who relied on it differently. In all five cases a single verb covers operations that differ in what they leave authoritative, what they leave recoverable, and what they leave a later reader able to check.

This essay adopts the history-primary view, on which the state of a system is a fold over an append-only record and is not itself the thing that is stored. On that view an operation that purports to remove something is not a deletion from the world but an operation defined on the record, either as an addition to it, as a change in how its entries are stored, or as a change to the entries themselves. The three possibilities can be told apart by a precise invariant, the sequence of commitments to the entries, and each has characteristic consequences for authority, for replay, and for audit.

The contributions are of three kinds. First, four operations are defined by their type and their effect, and the central observation is that authority, recoverability, and auditability are independent dimensions along which the operations differ, so that preserving a commitment to an entry is not the same as preserving its content or its former authority. Second, a conditional theorem states when accountability survives retraction: if the log is append-only, retractions are themselves entries, and the authority rule respects withdrawal, then a retracted claim leaves the authoritative set and every earlier state remains replayable. The theorem is elementary once the semantics are fixed, and the work lies in fixing them. Third, the conditions under which the guarantee fails are analyzed in three cases, namely dependence between claims, demands for erasure, and unannounced rewriting, and for each the essay states exactly what is preserved and what is lost.

The essay does not argue that retraction should always be preserved in the record, that erasure should be refused, or that any particular institution handles retraction well or badly. Whether content ought to be erased is a question for law and ethics, and the formalism takes no position on it. What the formalism supplies is a statement of consequences, so that a decision to erase, withdraw, or revoke can be made with knowledge of what the decision leaves in place. Section 2 fixes definitions, Section 3 defines the four operations, Section 4 states the central result, Section 5 analyzes the conditions under which it fails, Section 6 collects counterexamples and boundary cases, Section 7 specifies tests, Section 8 relates the account to the wider framework, and Section 9 lists open questions.

2 Preliminaries

Definition 1 (Log). A log is a finite sequence $L = (e_1, \dots, e_n)$ of entries drawn from a typed alphabet Σ . Prefixes are written $L_{\leq k}$.

Definition 2 (Authority fold). Let S be a set of states and $s_0 \in S$ an initial state. A step function $\delta : S \times \Sigma \rightarrow S$ induces $\text{State}(L) = \delta(\dots \delta(\delta(s_0, e_1), e_2) \dots, e_n)$. Present authority is a projection $\text{Auth}(L) \subseteq \text{Claims}$ of this state. The rule used throughout is fixed below.

Definition 3 (Replayability and auditability). A log is replayable if $\text{State}(L_{\leq k})$ is recoverable for every k . It is auditable if the set of entries ever appended, together with their order, is recoverable from L .

Three kinds of identity are kept apart. A claim identity $c \in \text{Claims}$ names a proposition and persists across withdrawal and reactivation. An assertion event is an entry in which a claim is asserted, and is identified by its position in the log. A justification identity ι names one recorded ground for a claim.

Definition 4 (Claim entries). The entries that concern claims are the following.

- An assertion $a(c; x)$ asserts the claim c with payload x , the text of the claim, which the fold does not read. The first assertion of c introduces c . A later assertion of c is a reactivation when c is not live and a restatement when c is live. In both cases the justifications of c are unchanged and all earlier entries remain in the log. An assertion of an erased claim is recorded but leaves the claim erased.
- A justification $j(\iota, c, P)$ records that the finite set $P \subseteq \text{Claims}$ of premises is a ground for c . It is admissible only if c and every member of P have been introduced earlier in the log

and the relation of premises to conclusions over active justifications stays acyclic once it is added. A justification with $P = \emptyset$ grounds a base claim.

- A withdrawal $w(c)$ of an introduced claim and a withdrawal $v(\iota)$ of an existing justification are decisions. The entry $v(\iota)$ is recorded together with the identifier of the claim that ι grounds. A withdrawn justification is not restored, and a new ground is recorded under a new justification identity.
- An erasure notice $\varepsilon(c)$ marks c as erased. Erasure is absorbing: an erased claim is never live again, whatever follows.
- An annotation carries information that the fold does not read.

Each entry is analyzed into three components. Its header consists of the kind of the entry and the identifier of the claim it concerns. Its fold-visible fields are the further components that the fold reads, namely the pair (ι, P) for $j(\iota, c, P)$ and the identifier ι for $v(\iota)$, and none for the other kinds. Its payload is everything the fold does not read, namely the text x of an assertion and the text of annotations. In examples, $a(c, \mathcal{J})$ abbreviates the assertion of c followed by one justification $j(\iota_i, c, P_i)$ for each $P_i \in \mathcal{J}$ with fresh identities, and $a(c)$ without a family denotes a reactivation. Payloads are suppressed when they play no role.

Definition 5 (Live claims). A claim c is live in L , written $c \in \text{Live}(L)$, if c has been introduced, $\varepsilon(c)$ does not occur in L , and the last entry among the assertions and withdrawals of c is an assertion. A justification is active in L if its entry occurs in L and $v(\iota)$ does not.

Definition 6 (Support and authority). Write $p \prec c$ if p is a premise of some active justification of c . By the admissibility condition $\prec$ is acyclic, and so it is well-founded on the finite set of claims of a log. The supported set $\text{Supp}(L)$ is defined by recursion on $\prec$: $c \in \text{Supp}(L)$ if and only if $c \in \text{Live}(L)$ and some active justification $j(\iota, c, P)$ has $P \subseteq \text{Supp}(L)$. The authority rule is $\text{Auth}(L) = \text{Supp}(L)$.

Two features are immediate from the definitions and are used repeatedly. First, $\text{Supp}(L) \subseteq \text{Live}(L)$, so the rule respects withdrawal: a claim that has been withdrawn, or erased, and not reactivated is not authoritative. Second, support is a function of the live set and the active justifications alone, and the justifications of a claim are consulted only when the claim is live. Authority therefore requires recorded grounds, and a live claim with no active justification is asserted but not authoritative. Recording a justification as an entry of its own keeps the dependency structure a function of the log, so that replay reproduces it exactly, and it allows a ground to be added or withdrawn without touching the claim. Keeping the three identities apart allows the operations of the next section to be stated without conflating them. Withdrawing a justification does not withdraw the proposition, and reactivating a claim is not the creation of a new evidential basis.

3 Four Operations

The four operations are defined as maps on committed logs, and each is assessed by its effect on three properties: present authority Auth , replay of earlier states, and audit of what was ever recorded. The operations on claims use the entries of Section 2. Revocation concerns authorizations, which need three further kinds of entry.

Definition 7 (Authorization entries). A grant $g(\alpha)$ records that the authorization α was conferred. An action $u(\alpha, x)$ records that x was done under α . A revocation $r(\alpha, \tau)$, appended at position j , ends α with effect from position $\tau \leq j$, and $r(\alpha)$ abbreviates $r(\alpha, j)$. An action $u(\alpha, x)$ at position k is admissible in a log L if $g(\alpha)$ occurs before position k and L contains no revocation $r(\alpha, \tau)$ with $\tau \leq k$. A revoked authorization is not conferred again, and a new authorization receives a new identifier.

Evaluating admissibility in L gives the status of an action as assessed at the end of the log, and evaluating it in the prefix $L_{\leq k}$ gives its status as believed at the time of the action. The two statuses agree for every action unless some revocation is backdated, that is, unless it has τ smaller than the position at which it was appended.

1. **Withdrawal of an assertion.** Appends $w(c)$, or $v(\iota)$ for a single ground. The claim leaves Live, or the ground becomes inactive, and in either case every claim all of whose derivations depended on the withdrawn item leaves Auth. The original entries remain in the log.
2. **Revocation of an authorization.** Appends $r(\alpha, \tau)$. Acts on permissions and not on claims, so by default its effect is prospective: with τ equal to the position of the entry, actions taken under α before the revocation keep their admissibility.
3. **Deletion of an object.** Splits into two cases: tombstoning, and removal. Tombstoning has two components that act on different properties. An erasure notice $\varepsilon(c)$ is an extension and acts on authority exactly as a permanent withdrawal does. A redaction replaces the content of records by headers and acts on recoverability alone. Removal takes the entry out of the sequence outright. Section 5.2 treats tombstoning, and Section 5.3 shows that removal is a divergence.
4. **Alteration of a record.** Replaces an entry e_i with e'_i without appending and without a marker. Together with the removal of an entry from the sequence, it is the operation that changes the sequence of commitments, which is the property Section 5.3 uses to separate it from the other three.

As maps on logs, the operations have the following types. Withdrawal sends L to $L \cdot w(c)$ or $L \cdot v(\iota)$, and revocation sends L to $L \cdot r(\alpha, \tau)$, so both are extensions. Tombstoning sends L to $(L \cdot \varepsilon(c))^T$ for a set T of positions, an extension followed by a redaction. Alteration sends L to the log in which the entry at some position i has been replaced, which is a divergence, and these terms are made precise in Section 5.3. The operations also differ in what they act on. Withdrawal acts on the present status of a past assertion, and so reaches backwards in the sense that every claim derived from the withdrawn one loses authority however long ago it was derived. Revocation acts on the future status of actions and leaves the status of earlier actions as it was, unless it is backdated. Redaction acts on the availability of content and not on its status. Alteration acts on the record itself.

Operation	Type	Authority	Replay	Audit
Withdrawal	Extension	The item and every claim dominated by it leave Auth	Preserved	Preserved
Revocation	Extension	No claim changes; later actions under α become inadmissible	Preserved; the as-believed statuses of past actions remain recoverable	Preserved
Tombstoning	Extension, then redaction	The notice removes the claim and every claim dominated by it; redaction alone changes nothing	Preserved from the notice onward; earlier authority of the erased claim and its dependents needs a checkpoint	Commitments, headers, positions and order; content only under custody
Alteration	Divergence	Unconstrained	Fails from the first altered position; detectable up to the latest anchor	Fails from the first altered position; detectable up to the latest anchor

Read by columns, the table shows that the three properties vary independently. Withdrawal changes authority and nothing else. Revocation changes the admissibility of future actions and nothing else. Redaction changes the availability of content and, without an erasure notice, nothing about authority. Alteration changes all three at once and without a trace. The consequence stated in the abstract follows: preserving a commitment to an entry preserves neither its content, which redaction removes, nor its former authority, which a withdrawal or an erasure notice removes. The table also orders the operations by how much of the record they leave intact: the first two leave it fully intact, the third weakens it in a controlled way, and the fourth breaks it.

4 The Central Result

Theorem 1 (Conditional accountability). *Let L be a log such that (i) L is append-only, (ii) every retraction, that is, every $w(c)$, $v(\iota)$ and $\varepsilon(c)$, is an entry of L , and (iii) the authority rule respects withdrawal, in the sense that $\text{Auth}(L') \subseteq \text{Live}(L')$ for every log L' under consideration. Then:*

1. *for every k , $\text{State}(L_{\leq k})$ is recoverable (replay);*
2. *the set of entries ever asserted is monotone nondecreasing in k (audit);*
3. *if $w(c) \in L$ and c is not asserted after $w(c)$, or if $\varepsilon(c) \in L$, then $c \notin \text{Auth}(L)$ (non-authority).*

Proof. For (1), $\text{State}(L_{\leq k})$ is the fold of the first k entries. Because L is append-only, the first k entries of every later presentation of L are the same entries in the same order, so the fold can be recomputed from the prefix at any time. For (2), let A_k be the set of entries asserted among $e_1, \dots, e_k$. Appending never removes an entry from the sequence, and a withdrawal is itself an entry and not a removal, so an assertion once recorded stays recorded and $A_k \subseteq A_{k+1}$. For (3), suppose first that $w(c)$ occurs at position j and that c is not asserted afterwards. At $L_{\leq j}$ the last entry among the assertions and withdrawals of c is a withdrawal, so c is not live. Every

later entry concerning c is a further withdrawal, and so c remains outside $\text{Live}(L)$. If instead $\varepsilon(c) \in L$, then $c \notin \text{Live}(L)$ directly from the definition of liveness. In both cases hypothesis (iii) gives $c \notin \text{Auth}(L)$. $\square$

Hypothesis (iii) is not redundant. An authority rule that kept a claim authoritative on the strength of its past assertion would satisfy (i) and (ii) and would violate the conclusion. The rule Supp of Section 2 satisfies (iii) by its definition, which is why the rule is fixed before the theorem and not after it.

The proof is short because the content of the theorem lies in its hypotheses and in what its conclusion leaves unsaid. The conclusion says nothing about claims that depend on the withdrawn one, and the append-only hypothesis fails under erasure and under rewriting. The three subsections that follow take these in turn.

5 Where the Condition Fails

5.1 Cascading dependence

The theorem shows that a withdrawn claim leaves the live set and, by hypothesis (iii), the authoritative set. It says nothing about claims that were asserted on the strength of the withdrawn one, and those claims are where retraction usually matters in practice. If authority were taken to be the live set, a log could contain a conclusion whose every recorded ground has been withdrawn, and the system would continue to treat that conclusion as authoritative. The rule of Section 2 closes this gap by connecting authority to justification while remaining a function of the log, and this subsection analyzes what the rule does.

A derivation of c in L is a finite tree whose root is labelled with the claim c , in which every node is labelled by a live claim d together with a chosen active justification ι of d , and in which the claims labelling the children of the node are exactly the premises of ι . A node whose chosen justification has no premises is a leaf.

Proposition 1 (Support is derivability). *$c \in \text{Supp}(L)$ if and only if c has a derivation in L . In particular, every claim in $\text{Auth}(L)$ is grounded, through a finite chain of recorded justifications, in live claims alone.*

Proof. By induction on $\prec$. If $c \in \text{Supp}(L)$, take an active justification with premise set $P \subseteq \text{Supp}(L)$, apply the inductive hypothesis to each premise, and attach the resulting derivations beneath a root labelled c . Conversely, the subtrees beneath the root of a derivation are derivations of the members of the premise set of a single active justification, hence those members lie in $\text{Supp}(L)$ by the inductive hypothesis, and c is live by the labelling condition. $\square$

Example 1 (Orphaned authority under the naive rule). Take the log $a(b, \{\emptyset\})$, $a(u, \{\{b\}\})$, $w(b)$. Then $\text{Live} = \{u\}$ while $\text{Supp} = \emptyset$. If the live set were taken as authoritative, u would remain authoritative although its only ground contains a withdrawn premise. Under the rule of Section 2, u remains in the log, remains live, and is not authoritative.

The next result states exactly which claims a single withdrawal removes, and it is the formal counterpart of the observation that retracting one premise should not remove a conclusion that has independent grounds.

Proposition 2 (Effect of a withdrawal). *(a) Let $c' \in \text{Live}(L)$ and $L' = L \cdot w(c')$. Then*

$$\text{Supp}(L) \setminus \text{Supp}(L') = \{c \in \text{Supp}(L) : \text{every derivation of } c \text{ in } L \text{ has a node labelled } c'\}.$$

(b) Let ι be an active justification in L and $L' = L \cdot v(\iota)$. Then $\text{Supp}(L) \setminus \text{Supp}(L')$ is the set of claims $c \in \text{Supp}(L)$ such that every derivation of c in L has a node whose chosen justification is ι .

Proof. In case (a) the live set of L' is $\text{Live}(L) \setminus \{c'\}$ and the active justifications are unchanged, so the derivations in L' are exactly the derivations in L with no node labelled c' . In case (b) the live set is unchanged and the active justifications lose ι , so the derivations in L' are exactly those in L that never choose ι . In both cases $c \in \text{Supp}(L')$ if and only if some derivation of c in L survives, by the previous proposition. $\square$

Example 2 (Independent grounds). Take the log $a(p, \{\emptyset\})$, $a(q, \{\emptyset\})$, $a(c, \{\{p\}, \{q\}\})$, $w(p)$. Then $\text{Supp} = \{q, c\}$, because c retains the derivation through q . A subsequent $w(q)$ removes c . Withdrawing instead only the justification of c that cites p , by $v(\iota_p)$, has the same effect on authority and leaves the proposition p itself in place. The claim c is neither fragile nor immune, and its authority tracks whether any recorded ground survives.

Two further properties follow directly from the characterization. First, Supp is monotone in the live set and in the set of active justifications: if both grow, every derivation in the smaller log is a derivation in the larger, so Supp grows. Second, Supp is determined by the live set together with the active justifications, so any two logs that agree on both have the same authoritative set. The second property has a consequence that distinguishes two ways of implementing the rule, and a third reading must be excluded.

Under lazy propagation, the rule is applied as a derived projection, and a withdrawal appends no entries on behalf of dependent claims. Under eager propagation, entries are appended for the dependents that lose support, and what authority does afterwards depends on the kind of entry appended. If the appended entries are ordinary withdrawals $w(d)$, they are decisions, attributable to an author and independent of the premise whose loss prompted them. If they are suspension annotations recording mechanically that d lost support at a given position because of a given withdrawal, they are not read by the fold, and authority remains Supp .

Example 3 (Restoration). Consider three logs that share the prefix $a(p, \{\emptyset\})$, $a(c, \{\{p\}\})$, $w(p)$ and are followed by the reactivation $a(p)$. Under lazy propagation nothing is appended on behalf of c , the live set returns to its original value, and $\text{Auth} = \{p, c\}$ without any further act. Under eager propagation with decisions, the log also contains $w(c)$ after $w(p)$, so after the reactivation of p the claim c is not live and stays outside Auth until c is itself reactivated. Under eager propagation with suspension annotations, the annotation is not read, so the outcome is the same as under lazy propagation, with an additional audit record of why c was unsupported in the interval.

The distinction matters because the restoration of a premise ought not to reverse an independent decision, whereas it may reasonably reverse a mechanical consequence. Recording a consequence as a decision turns a mechanical event into a judgment attributed to someone, and makes it irreversible by the mechanism that produced it. Recording it as an annotation preserves the cause for audit without altering authority. A system that must attribute every loss of authority to an actor should record decisions only where a decision was made and should use annotations for mechanical consequences, so the two are different types of entry and not different degrees of eagerness.

The guarantees above are relative to the recorded justification structure, and this is the main limit of the rule. A claim that in fact relied on c' but was recorded without declaring that reliance is invisible to propagation.

Example 4 (Undeclared dependence). Take the log $a(p, \{\emptyset\})$, $a(d, \{\emptyset\})$, $w(p)$, where the author of d relied on p but recorded d as a base claim. Then $d \in \text{Auth}$ after the withdrawal, and no rule defined on the log can remove it.

For scholarly settings this means that the dominant failure of retraction is a failure of declaration and not of propagation. A citation graph is not a justification graph: a paper may cite a retracted article as background, as a point of contrast, or as evidence, and only the last role makes the cited article a premise. The checker described in Section 7 should therefore report two things separately, namely the dependents lost under the declared justification structure, and the citation edges that were not declared as premises and so remain candidates for manual audit.

Two remarks complete the section. If the acyclicity condition were dropped, mutually supporting claims could be recorded, and a greatest-fixed-point reading of support would leave such a pair authoritative after the base claims beneath them were withdrawn, whereas the least-fixed-point reading used here leaves neither. Acyclicity is what excludes self-supporting loops. Finally, maintaining Supp incrementally is an instance of truth maintenance and of deletion in incremental view maintenance. By keeping, for each claim, a count of the active justifications currently satisfied, a withdrawal can be processed with work proportional to the part of the graph incident to the claims that lose support, which is what makes the rule practical at the scale of a repository or a literature.

5.2 Erasure demands

Privacy-driven deletion conflicts with accountability because the two demands ask opposite things of the same entry. The first asks that its content cease to be available, and the second asks that it remain possible to say what happened and to check any later statement about it. A log that supports retraction only by appending cannot satisfy the first demand at all, since nothing is removed. Satisfying it requires an operation that changes records, and the question is how much of the log’s structure can survive that change. The answer developed here is that content can be separated from commitment, and that the effect of erasure on authority can be separated from its effect on content. What a log must retain in order to remain checkable is much less than what an entry says.

Definition 8 (Committed log). Fix a hash function H and an injective, prefix-free encoding enc into bit strings. Each entry e_i is analyzed into its header η_i , its fold-visible fields φ_i , and its payload π_i , as in Section 2. When the entry is appended, two random strings s_i and t_i of k bits each are drawn, and the entry is committed through two leaves and a root:

$$f_i = H(s_i \parallel \text{enc}(\varphi_i)), \quad p_i = H(t_i \parallel \text{enc}(\pi_i)), \quad h_i = H(\text{enc}(\eta_i) \parallel f_i \parallel p_i).$$

Chain digests are defined by a fixed constant g_0 and $g_i = H(g_{i-1} \parallel h_i)$, and the head g_n is anchored outside the log by a means that authenticates it, such as publication or a signature. The header enters h_i in the clear, so the chain authenticates it, and the two leaves allow the fold-visible fields and the payload to be hidden independently while remaining committed.

Definition 9 (Redaction and tombstone). A redaction of position i discards openings and keeps the header and the leaves. A payload redaction discards t_i and π_i and retains η_i , φ_i , s_i and the leaf p_i . A claim redaction discards s_i and φ_i as well, and retains only η_i and the leaves f_i and p_i . It is admissible only if $\varepsilon(c)$ occurs in the log for the claim c named in η_i . The discarded randomness and content are destroyed or placed in custody, as discussed below. The erasure

notice $\varepsilon(c)$ is never redacted, since its entire content is its header. For a set T of positions, write L^T for the log in which the positions in T are redacted by admissible redactions. A tombstone is a redacted record. Claim identifiers are opaque labels assigned at first assertion, so a header reveals nothing about the text of the claim.

The admissibility rule answers the question of which semantic metadata survives redaction. The header of every entry, and the fold-visible fields of every entry concerning a claim that is not erased, always survive, so the fold remains defined on the redacted log. Because the header enters h_i in the clear and the surviving fields are checked against their leaf, a tombstone cannot be given a different header, and a surviving field cannot be altered, without changing the head. For an erased claim the fold needs only that the claim is erased, because its justifications are consulted only when it is live, and so they can be removed without leaving the support recursion undefined.

The chain digests are computed from the commitments h_i alone, and this is what allows redaction to leave the remainder of the log verifiable.

Proposition 3 (Integrity under redaction). *Suppose enc is injective and prefix-free and the head g_n is authentic. For every admissible T , the head recomputed from the retained headers and leaves of L^T equals g_n , and every retained opening verifies against its leaf. Conversely, if a log L' has the same authentic head as L and every opening it keeps is consistent with its leaf, then the headers of L' coincide with those of L and every component that L' keeps open has the same content as in L , or a collision in H has been found.*

Proof. The first claim is the definition of the chain, since h_i is a function of η_i , f_i and p_i alone, and the retained openings are consistent with their leaves by construction. For the converse, equal heads imply equal commitment sequences unless the chain itself contains a collision. Because enc is prefix-free and the leaves have fixed length, the input to h_i parses uniquely, so equal commitments imply equal headers and equal leaves unless a collision occurs. A component opened in L' with content different from that in L and the same leaf is then a collision at the leaf. $\square$

A head-preserving change to a log can therefore change which components are open, but it cannot substitute content for content or alter a header, up to the collision resistance of H . This is the formal separation between redaction and the silent rewrites of Section 5.3: alteration changes the head, and redaction does not. The next two results separate what a redaction does to authority from what an erasure notice does.

Proposition 4 (Redaction is authority-neutral). *Let T be admissible and let E_T be the set of claims concerned by claim redactions in T . For every k that is at least the position of $\varepsilon(c)$ for every $c \in E_T$, $\text{Auth}(L_{\leq k}^T)$ is computable from L^T and equals $\text{Auth}(L_{\leq k})$. If T contains only payload redactions, this holds for every k .*

Proof. Payload redactions retain every fold-visible field, so the fold reads the same inputs in L^T as in L . For a claim $c \in E_T$ and k at or beyond the position of $\varepsilon(c)$, the claim is not live in $L_{\leq k}$ and is not live in $L_{\leq k}^T$, since the notice is retained, and a claim that is not live is not consulted for its justifications. The entries of every other claim are unchanged. Hence Live and the active justifications of the claims that matter coincide, and so do the supported sets. $\square$

Proposition 5 (Erasure notice is withdrawal). *Let $c \in \text{Live}(L)$. Then $\text{Auth}(L \cdot \varepsilon(c)) = \text{Auth}(L \cdot w(c))$. Consequently the claims that lose authority through the erasure of c are exactly those all of whose derivations in L pass through a node labelled c .*

Proof. Both logs have live set $\text{Live}(L) \setminus \{c\}$ and the same active justifications, so the supported sets coincide. The second sentence is part (a) of the effect-of-a-withdrawal result of Section 5.1. $\square$

The two differ only in what follows them: a reactivation restores c after $w(c)$, and nothing restores it after $\varepsilon(c)$. An erasure notice never enlarges authority, since it only removes a claim from the live set, and by the previous proposition the redaction that follows it changes nothing. Persistence of the erased status is carried by the notice, which is an entry, so it does not depend on which records are open: a record that is later disclosed does not resurrect the claim.

What a redacted record discloses and what it takes away depends on three things that must be kept apart, namely the computational hiding of the commitment, the existence of retained openings, and actual deletion.

Proposition 6 (Hiding, witnesses, and deletion). *Let i be a redacted position, let e_i denote a component that was hidden there, either the fold-visible fields or the payload, let r_i denote its randomness and $\lambda_i = H(r_i \parallel \text{enc}(e_i))$ its leaf, and let x be a candidate for e_i that is consistent with the header and the surviving components.*

1. (Computational hiding.) *In the random-oracle model, if r_i is uniform on k bits, independent of the candidates, and unknown to the adversary, then an adversary that makes at most q queries to H distinguishes any two candidates for e_i with advantage at most $q2^{-k}$.*
2. (Retained witnesses.) *Any party that holds, or has ever held, a pair (r_i, e_i) consistent with λ_i can open the component and verify any candidate. The log cannot show that no such party exists.*
3. (Deletion.) *That every opening has been destroyed is a fact about the world and not about the log. The log can carry at most an attestation of destruction by a custodian, which is a claim whose authority is governed by Section 2 and whose grounds lie outside the log.*
4. (Weak randomness.) *If r_i is absent, public, or of low min-entropy, then any party able to enumerate a candidate set X for e_i recovers e_i with at most $|X|$ evaluations of H for each possible value of the randomness, so the commitment hides nothing beyond the searcher's budget.*

Proof. Item (1) is the standard bound for hash commitments in the random-oracle model. The output of H on a string beginning with r_i is independent of e_i unless the adversary queries a string that begins with r_i , and by a union bound this happens with probability at most $q2^{-k}$. The statement is model-dependent, and it supports hiding only against adversaries who lack the opening and hold no side information correlated with it. Items (2) to (4) follow from the definitions. $\square$

A salt that remains stored in the tombstone does not repair item (4), because a public salt only prevents precomputation across entries. Hiding requires that the randomness be secret, so the question of erasure becomes a question about custody of r_i and about what has actually been destroyed.

Custody of r_i	Parties able to open	What accountability retains
Destroyed everywhere, as attested	None, if the attestation is true	Existence, position, order, header and commitment; no claim about the content can be checked, and the destruction itself rests on the custodian's attestation.
Held by the subject or original author	The holder, at the holder's discretion	The above, plus checkability that depends on the cooperation of one party.
Escrowed with custodians under a release rule	The custodians, when the stated condition is met	The above, plus checkability that does not depend on the subject's cooperation, at the cost that the content remains recoverable in principle.

These regimes are points on one axis, and which of them counts as deletion is determined by what is actually destroyed and by who holds what, and not by the structure of the log. Destroying the randomness is the log-level analogue of destroying an encryption key to render encrypted data unrecoverable. The log has no way to tell a reader which regime was applied unless an entry records it, and such an entry is itself a claim made by a custodian. A redacted record whose commitment is satisfied by a presented opening can be disclosed, for example when custodians release an escrowed opening. Disclosure does not alter authority, since authority is determined by the entries, including erasure notices, and not by which records are open. It is classified separately in Section 5.3.

The effect of an erasure notice on authority is the same cascade as a withdrawal, and it exposes a cost that erasure inherits from propagation. The set of claims dominated by c can be large, so a request that concerns a single entry can remove authority from claims far from it in the dependency graph. Three responses are available: to refuse or review erasure of claims whose dominated set is large, to accept the cascade, or to require that dependents be re-justified before the notice is appended, by recording justifications for them that do not pass through c , which the entries of Section 2 permit directly.

Redaction also changes what can be recomputed from the log. A payload redaction affects no authority at any prefix. A claim redaction leaves authority computable at the prefixes at or after the erasure notice, and it removes the means to recompute, at earlier prefixes, the authority of the erased claim and of the claims that depended on it. Clause (1) of the theorem therefore holds for the redacted log from the erasure notice onward, and for earlier prefixes only if a checkpoint was kept. Clause (2) holds in a weakened form, in which commitments, headers, positions and order are recoverable but contents are recoverable only under custody. If the question of what was treated as authoritative at a given time must remain answerable, a checkpoint entry holding the opaque identifiers of $\text{Auth}(L_{\leq k})$ for the affected positions can be appended before the claim redaction. A checkpoint preserves what was authoritative and when, without preserving why.

Finally, every statement in this subsection is relative to one replica. A replica that keeps the open record and its randomness holds the content, and for that replica a tombstone elsewhere is only a redacted view. Erasure in the stronger sense, that the content is available nowhere, requires every replica holding the record to redact it, which is a coordination problem across systems with no shared log and is deferred to the open questions.

5.3 Silent rewrites

The first two subsections kept the log's integrity intact: retraction appends, and redaction leaves every head unchanged. The remaining case is a transformation that changes recorded history without announcing it. An obvious definition of alteration is a map under which $\text{State}(L'_{\leq k}) \neq$

$\text{State}(L_{\leq k})$ for some k , but Section 5.2 showed that a claim redaction also changes what can be replayed, so a definition through State does not separate alteration from erasure. The invariant that does separate them is the sequence of prefix heads.

Compare a log L with commitments $h_1, \dots, h_n$ and a log L' with commitments $h'_1, \dots, h'_m$, and let p be the length of their longest common prefix of commitments. There are four cases by commitments. In an extension, $p = n < m$ and L' is L followed by further entries, up to which components of the first n records are open. In a commitment-equal pair, $p = n = m$, so the commitments agree and the logs can differ only in which components are open. In a truncation, $p = m < n$ and the tail of L is missing. In a divergence, $p < \min(n, m)$, so some record within both logs has a different commitment, whether by replacement, insertion, or removal from the middle of the sequence. Alteration is defined as divergence.

Commitment identity does not determine the direction of change. Within a commitment-equal pair, let $\text{open}(L)$ be the set of pairs of a position and a component, the fold-visible fields or the payload, that are open and consistent with their leaves. The change from L to L' is a redaction if $\text{open}(L') \subsetneq \text{open}(L)$, a disclosure if $\text{open}(L) \subsetneq \text{open}(L')$, a re-representation if the two sets are incomparable, and the identity if they are equal. A disclosure is not a redaction in reverse, and neither is an alteration, because every open component in either log verifies against the same leaf.

Proposition 7 (Classification by prefix heads). *Up to collisions in H : in an extension or a commitment-equal pair, $g_k(L') = g_k(L)$ for every $k \leq n$. In a truncation, $g_k(L') = g_k(L)$ for every $k \leq m$, but $g_n(L)$ cannot be reproduced from L' . In a divergence with first differing position $i = p + 1$, $g_k(L') \neq g_k(L)$ for every $k \geq i$ with $k \leq \min(n, m)$.*

Proof. The first two statements hold because g_k is a function of $h_1, \dots, h_k$ alone. For the third, $g_i = H(g_{i-1} \parallel h_i)$ and $g'_i = H(g_{i-1} \parallel h'_i)$ with $h_i \neq h'_i$, so $g_i \neq g'_i$ unless a collision occurs. For $k > i$ the inputs $g_{k-1} \parallel h_k$ and $g'_{k-1} \parallel h'_k$ differ in their first component, so the outputs differ unless a collision occurs, and the claim follows by induction. $\square$

Operation	Prefix heads $g_k, k \leq n$	Content of old records
Withdrawal, revocation, erasure notice (extension)	Unchanged; new heads are added	Unchanged
Redaction	Unchanged	Removed, with a marker in the header
Disclosure	Unchanged	Restored, verified against the commitment
Alteration (divergence)	Changed from the first altered position onward	Substituted or spliced out, without a marker
Truncation of the tail	Unchanged for the kept prefix; the anchored head is lost	Tail records absent

This gives a separation that does not depend on replayed state. Withdrawal, revocation and the erasure notice extend the log, redaction and disclosure re-present it, and alteration diverges from it, and the cases are distinguished by what happens to the sequence of prefix heads. It also settles the second form of deletion distinguished in Section 3: removal of an entry from the sequence is a divergence, or a truncation when the removed entries lie at the tail, so tombstoning is the only form of deletion that remains inside the family of operations that preserve accountability.

Whether a given alteration can be detected depends on what a verifier holds besides the log. An anchor is a head $g_m(L)$ that was recorded outside the log, and authenticated, at the time the log had length m , for instance by publication or by a copy held by another party.

Proposition 8 (Detection by anchors). *Let a verifier hold anchors $g_{m_1}(L), \dots, g_{m_s}(L)$ with $m_1 < \dots < m_s \leq n$, and let the verifier accept a presented log L' if and only if L' has length at least m_s and $g_{m_j}(L') = g_{m_j}(L)$ for every j . Then, up to collisions in H : a divergence with first differing position i is rejected if and only if $i \leq m_s$; a truncation to length m is rejected if and only if $m < m_s$; and when a divergence is rejected, the least j at which the check fails places i in the interval $(m_{j-1}, m_j]$.*

Proof. For $m_j < i$ the prefixes agree, so the check at m_j passes. For $m_j \geq i$ it fails by the classification above. The statement about truncation is the same argument applied to the length condition, and the localization follows from taking the least failing j . $\square$

Two consequences follow. First, a divergence or a truncation that lies entirely after the most recent anchor is undetectable until a later anchor covers it, so the interval between anchors is the window of vulnerability and is a design parameter and not a property of the log. Second, the append-only hypothesis of the theorem in Section 4 is an operational claim about the storage, and its verifiability is bounded by anchoring. Clauses (1) and (2) of the theorem are guarantees about the anchored prefix of the log, and for the unanchored suffix they rest on trust in whoever holds the storage.

The version-control case is a direct instance. Commit identifiers in a repository are digests that include the identifiers of their parents, so rewriting a commit changes the identifier of every descendant, which is the divergence property. A force-push is an alteration in this sense, and the anchors are the clones, tags and signed references held elsewhere. A repository whose history has been force-pushed is detected as altered by exactly those parties who hold an anchor at or after the rewritten commit.

Finally, alteration is never necessary for correction. An erroneous claim can be corrected by appending a withdrawal of it and an assertion of a corrected claim, which keeps the record that the error existed. The three identities of Section 2 make the two kinds of correction explicit. When the correction is a restatement of the same proposition, a typographical repair for example, the corrected payload is recorded as a new assertion event under the same claim identity, and dependents keep their grounds. When it is a substantive correction, the corrected claim has a new identity, and the dependents of the old claim do not silently transfer: each moves by recording a new justification that cites the corrected claim, so the transfer is an explicit act with an author. Alteration is therefore a convenience that trades away replay and audit, and the framework should treat it as the operation whose only legitimate use is to be refused or detected.

6 Counterexamples and Boundary Cases

Each case below is stated as a log, the status it should receive under the rules of the preceding sections, and the clause of the theorem of Section 4 that it stresses. Clause (1) is replay, clause (2) is audit, and clause (3) is non-authority of a withdrawn claim that is not asserted again.

6.1 A retraction that is later retracted

Take the log $a(p, \{\emptyset\})$, $a(q, \{\emptyset\})$, $a(c, \{\{p\}\})$, $w(p)$, $w(c)$, $a(c)$. The final entry reactivates c . Clause (3) does not apply, because its hypothesis excludes a later assertion, and the theorem is not contradicted. After the reactivation c is live, but its only active justification has the premise p , which is not supported, so $c \notin \text{Supp}$ and c is reported as asserted but ungrounded. Suppose now that the author, intending c to rest on q , appends $j(\iota', c, \{q\})$. Then c has an active justification whose premise is supported, and $c \in \text{Supp}$.

The case tests the distinction that Section 2 draws between reactivation of a claim and the creation of a new evidential basis. Reactivation restores liveness and not support, and the system correctly reports the claim as dormant until a ground is recorded. A model that fixed the justifications of a claim at its first assertion would leave c unsupported despite the author's intent, with no remedy short of alteration. A model that treated every re-assertion as a new claim would silently detach the dependents of c . The three identities avoid both outcomes.

6.2 Concurrent retractions on a branched history

Let two branches extend a common prefix L with sequences of entries E_1 and E_2 . Assume that claim and justification identities first introduced in a branch are fresh, so that no identity is introduced in more than one branch. A naive merge interleaves the entries of E_1 and E_2 and reads the result by the rules of Section 2.

If the branches concern disjoint sets of claims, where a branch concerns a claim if it contains an assertion, a withdrawal, an erasure notice, or a justification of that claim, and the union of the premise relations over active justifications in L and both branches is acyclic, then every interleaving yields the same live set and the same supported set. The liveness of a claim is fixed by the last entry among its assertions and withdrawals together with the absorbing erasure notice, all of which lie within a single branch. The active justifications of a claim are determined by its justification entries and their withdrawals, which also lie within a single branch. Hence Live and the active justifications do not depend on the interleaving, and Supp is a function of them.

Disjointness does not make the merge free of conflict. Suppose $c \in \text{Live}(L)$, the first branch appends $w(c)$, and the second appends $a(d, \{\{c\}\})$. In either order the merged log has c withdrawn and d live, so $d \in \text{Live}$ and $d \notin \text{Supp}$. The author of each branch saw a consistent history, and the inconsistency exists only in the merge. Under the rule of Section 2 the claim d is correctly not authoritative, and a merge procedure can report the claims in $\text{Live} \setminus \text{Supp}$ that were supported on one of the branches as conflicts for review.

When both branches concern the same claim, the interleaving matters, and the order-independent outcome cannot be obtained from any interleaving under the rule that the last entry decides. Suppose $c \in \text{Live}(L)$, the first branch appends $w(c)$, and the second appends $w(c)$ followed by $a(c)$. The interleaving $E_1 E_2$ leaves c live and the interleaving $E_2 E_1$ leaves it withdrawn. An explicit merge operation is therefore needed.

Definition 10 (Merge). The merge of branches $E_1, \dots, E_s$ over L is the log $L \cdot E_1 \cdots E_s \cdot M$, where M is a sequence of resolution entries in canonical order by identifier. M contains $w(c)$ for every claim c that is concerned by some branch and is not live in $L \cdot E_k$ for some branch E_k that concerns it, and it contains $v(\iota)$ for every justification recorded in the branches that lies on a cycle of the combined premise relation over active justifications.

Proposition 9 (Merge is order-independent). *For every permutation of the branches, the merged log has the same live set, the same active justifications, and hence the same supported set. A claim concerned by at least one branch is live in the merge if and only if every branch that concerns it leaves it live.*

Proof. Let c be concerned by the nonempty set B_c of branches. If some branch in B_c leaves c not live, then M contains $w(c)$ after all branch entries, so c is not live in the merge in every order. If every branch in B_c leaves c live, then none of them contains $\varepsilon(c)$. If no branch in B_c contains an assertion or a withdrawal of c , then c is live in L and stays live. Otherwise the last assertion or withdrawal of c in the merged log lies in the last branch of B_c that contains one, and it is an assertion because that branch leaves c live, so c is live in every order. Claims concerned by no branch keep their status in L . Justification entries are additive and withdrawals of justifications are absorbing, so the set of active justifications is a function of the set of entries and not of their order, and M is a function of the set of branches and not of their order. Hence Live and the active justifications are order-independent, and Supp is a function of them. Finally the premise relation over active justifications is acyclic after the resolution entries: the relation of L alone is acyclic, so every cycle of the combined relation contains an edge from a branch justification, every branch justification lying on a cycle has been withdrawn, and any cycle remaining after the withdrawals would have been a cycle before them. $\square$

The merge resolves conflicts toward the smaller authority set, which is the safe direction, since by Section 5.2 an erasure notice and a withdrawal never enlarge authority. Resolution entries are ordinary withdrawals, so the merged log is an ordinary log and no new state is introduced. The case stresses clause (1): the state that the author of one branch replayed is not a prefix state of the merged log, so replay holds branch by branch and not for the merge as a whole.

6.3 A revocation whose effects were already relied upon

Take the log $g(\alpha)$, $u(\alpha, x)$, $r(\alpha, 2)$, where $u(\alpha, x)$ is at position 2 and the revocation, appended at position 3, is backdated to position 2 because the authorization is later found to have been compromised from that point. Evaluated in the prefix, the action was admissible, which is its status as believed at the time. Evaluated in the full log it is inadmissible, which is its status as assessed. Both statuses are recoverable because the backdating is an appended entry and nothing about the earlier state was changed. The alternative, editing the log so that the action was never admissible, is an alteration, and it destroys the record that the system believed the action to be authorized, which is precisely what an investigation into reliance on the action would need. The case stresses clause (2): audit requires that the as-believed status remain recoverable alongside the as-assessed one. It also shows that the choice of τ is itself a consequential judgment, so the entry should record its author, a field that the minimal model omits.

6.4 A withdrawal that arrives after the claim has been cited elsewhere

Let a second log L_B contain an assertion whose justification cites a claim c that is live in a first log L_A , and let L_A later append $w(c)$. Within L_A the theorem applies and c is not authoritative. In L_B nothing has been recorded, so a fold over L_B alone still counts c as live. The theorem is silent here because clause (3) is a statement about a single log. Propagation across the boundary needs an import entry in L_B that records the remote claim together with an anchor, a head of L_A at the time of import. The importing system can then refresh the status of c by

obtaining a current presentation of L_A , checking by the criterion of Section 5.3 that it extends the anchored one, and evaluating authority there. Between refreshes the authoritative set of L_B is stale relative to L_A , which is the same window-of-vulnerability structure that anchors have in Section 5.3. The case stresses clause (3), and the question of how freshness should be specified is taken up in Section 9.

7 Implementable Tests

The checker described here is a specification of inputs and outputs, with no commitment to an implementation. Its purpose is to make each proposition of the preceding sections testable on concrete logs.

Inputs. A log file, with one record per line in log order, giving the position, the kind of entry (assertion, justification, withdrawal of a claim, withdrawal of a justification, erasure notice, annotation, grant, action, revocation, or checkpoint), the identifiers of the claim, justification or authorization concerned, the premise set for a justification, the effective position for a revocation, the header, the two leaves and the commitment, and, for each of the fold-visible fields and the payload, either the open component with its randomness or a marker that it has been redacted. An optional anchors file listing pairs of a length and a head. An optional second log, presented as a later or alternative version of the first. An optional file of citation edges between claim identifiers, each marked as a declared premise or not.

Outputs. The checker reports (i) the authoritative set $\text{Auth} = \text{Supp}$, the live set, and the dormant claims in $\text{Live} \setminus \text{Supp}$, each with the withdrawn or erased premises whose loss removed its support; (ii) the result of recomputing the chain from the commitments and checking every open component and every anchor, with failures localized to the interval between consecutive anchors as in Section 5.3; (iii) when a second log is supplied, its classification relative to the first as an extension, a redaction, a disclosure, a re-presentation, a truncation, or a divergence, with the first differing position; (iv) for every action, its status as believed and as assessed, with the actions on which the two differ; (v) the violations of admissibility, namely redactions that remove fold-visible fields of a claim with no erasure notice and justifications that create a cycle; and (vi) when citation edges are supplied, the edges from a live claim to a withdrawn or erased claim that were not declared as premises, which are the candidates for manual audit.

Property tests. Each proposition yields a test that can be run on randomly generated logs.

Property	Source	Check
Support is derivability	Section 5.1	Compute Supp by the recursion and by explicit search for derivations, and require equality.
Effect of a withdrawal	Section 5.1	For each live claim and each active justification, compare the claims removed by appending its withdrawal with the claims all of whose derivations pass through it.
Erasure notice is withdrawal	Section 5.2	Append $\varepsilon(c)$ and $w(c)$ to copies of a log and require the same authority.
Redaction is authority-neutral	Section 5.2	Apply random admissible redactions and require the same authority at every prefix at or after the relevant erasure notices; require the checker to reject inadmissible redactions.
Integrity under redaction	Section 5.2	Redact random admissible sets of positions and require that the recomputed head is unchanged and that all open components verify.
Classification	Section 5.3	Apply an extension, a redaction, a disclosure, a truncation, and a divergence to a log, and require the classification and the first differing position to match.
Detection by anchors	Section 5.3	Place anchors at random positions, alter or truncate at random, and require acceptance, rejection, and the localizing interval to be as the proposition predicts.
Merge	Section 6.2	Merge random branches in every order and require identical live and supported sets, equal to the stated conservative rule.

Version-control histories. A repository can be read as a log in which commit identifiers play the role of chained commitments, since each identifier is a digest that includes the identifiers of its parents. Comparing the tip a verifier recorded earlier with the current tip classifies the change: if the earlier tip is an ancestor of the current tip, the change is an extension; if the current tip is an ancestor of the earlier tip, it is a truncation; and otherwise it is a divergence beginning after the merge base. A force-push is therefore reported as a divergence or a truncation. A repository has no native redaction, because a commit identifier covers the content directly and removing content changes every identifier downstream. Every attempt to remove content from a repository’s history is accordingly reported as a divergence, which is a precise sense in which version control offers no accountable deletion. The anchors are the clones, tags and signed references held by other parties.

Citation graphs. Taking claims to be publications, withdrawals to be retraction notices, and declared premises to be the works relied on as evidence, the checker reports the dependents that lose authority under the declared structure and, separately, the citation edges to retracted works that were not declared as premises. The second list is a set of candidates and not a verdict, since a citation edge does not state whether the cited work was relied on, and extracting roles from text is outside the specification.

The checker does not judge content and does not decide policy. It computes what the log entails under the rules above, and it reports where the log is silent.

8 Relation to the Wider Framework

The account given here refines a distinction drawn in earlier work on replayable history. *The Log, Not the Wall* separates causal modification of a system, which is compatible with replayable history, from retroactive erasure, which is not, and it requires that history be appended and never replaced. Retraction by appending is a case of the first kind. The typology of Section 5 divides the second kind into three and shows that the three are not equally destructive. Alteration is retroactive erasure in the full sense: it changes the sequence of commitments, and so breaks replay and audit from the first altered position onward unless an anchor intervenes. Truncation is a limiting case that damages only the unanchored tail. Tombstoning is a third thing, an extension that records the erasure together with a change in which content is stored that leaves the commitments in place, and the results of Section 5.2 are that it weakens replay and audit in a precisely describable way and not wholesale. The statement that retroactive erasure destroys replayability is therefore true of divergence, and it needs qualification for redaction.

Commitment Before Appearance treats a single committed history as the source of two parallel reductions, a physical fold and a semantic fold, and places the boundary of history at the point where an observation is attested and accepted. In that vocabulary, the authority fold of Section 2 is a semantic reduction of the committed sequence, and the view that a reader sees, with some records open and others tombstoned, is a second reduction of the same sequence whose appearance can change while the commitments do not. The checkpoint entry proposed in Section 5.2 plays a role analogous to an attestation: a derived status is admitted into the history, so that what was authoritative at a given position remains available after the fold over the redacted log would yield something different.

Layering the World distinguishes rendered appearance, operative state, and historical inscription, and holds that inscription is authoritative at commit while state is authoritative between commits. Withdrawal and revocation act on the second layer: they change the operative status of claims and actions while the inscription remains intact. Redaction acts at the boundary between appearance and inscription, changing what is shown while leaving the inscription's commitments in place, and alteration is an attack on the inscription itself. The conclusion of Section 5.3 that corrections should be appended and not patched is an instance of the discipline in that essay by which a replay divergence is appended and never overwritten.

The Reversibility Gate also distinguishes four operations, namely revision, retraction, roll-back, and repair, but for a different purpose. The question there is when an action under an unsettled claim may be permitted, given how reversible the action is. The typology in this essay asks what each removal operation leaves in the record, and the two should be read as answering different questions. A correspondence between them is not asserted here.

What this essay adds to that body of work is a typology of removal operations stated through a single invariant, the sequence of commitments, together with the observation that authority, recoverability and auditability are independent dimensions along which the operations differ. It also adds the exact conditions under which accountability survives each operation: the append-only, retractions-as-entries and respects-withdrawal conditions for the theorem of Section 4, the declared justification structure for propagation, the admissibility of redaction and the custody of the commitment randomness for erasure, and anchoring for the detection of rewrites.

9 Open Questions

The analysis leaves a number of questions open, and several were met more than once in the preceding sections.

Supersession and correction. Section 2 separates claim, assertion and justification identities, which settles the questions about re-assertion and late justification that arise when the family of grounds is fixed at first assertion. What it leaves open is supersession. A correction that replaces a claim by a new one moves dependents only through explicit new justifications, and it is open whether supersession should be an entry with its own semantics, so that a dependent can inherit the grounds of the corrected claim by declaration, and what derivations should then record. It is also open whether a reactivation should be required to carry a fresh ground when the grounds that originally supported the claim have since been withdrawn.

What can stay hidden. Under the admissibility rule of Section 5.2, the fold-visible fields of the entries concerning a claim that is not erased are never redacted, so the dependency structure of live claims is public even when their content is not. The shape of that structure may disclose what the text does not. The question is whether authority can be computed from commitments to justification sets together with proofs of support, without disclosing the sets, and what such a scheme would have to assume.

Graded withdrawal. Withdrawal is binary in the account above, whereas corrections, expressions of concern, and downgrades of confidence are common in practice and are partial. A graded treatment would replace the two statuses by a partially ordered set of statuses and would take the status of a conclusion to be a monotone function of the statuses of its premises. The question is whether the characterization of Section 5.1, in terms of the claims all of whose derivations pass through the withdrawn claim, has an analogue in that setting, for example in terms of the premises whose status bounds that of a conclusion.

Systems with no shared log. Section 6.4 introduced import entries with anchors, and Section 5.2 noted that a tombstone is local to a replica. Both are instances of one problem, the coordination of retraction and erasure across systems that share no log. What guarantees are possible when the remote log cannot be polled, how freshness should be specified when it can, and what a replica must do to honor an erasure that originated elsewhere are all unsettled.

Merge rules. Section 6.2 defines an explicit merge operation that resolves conflicts toward the smaller authority set. It is open whether this operation is determined by natural requirements, such as independence of order, grouping, and repetition of branches together with the requirement that a merge not enlarge authority beyond what some branch grants, and what changes if some conflicts are resolved toward larger authority by explicit agreement, which would require a resolution entry authored by a party with standing.

Scale and incomplete declaration. Section 5.1 sketched incremental maintenance of Supp whose cost is proportional to the region affected by a withdrawal. Whether that region is small in practice depends on the shape of real dependency structures, such as citation networks and package ecosystems, and on how much of the true dependence is declared. The checker of Section

7 would allow both quantities to be measured on existing data. The answer determines whether erasure by cascading removal is practical, or whether dominated sets are in practice large enough to make re-justification before erasure, the third response of Section 5.2, a necessity.